All the major frameworks in the Spring ecosystem offer both non-reactive and a reactive options, including Spring Security, Spring Data, and Spring Cloud. Overall, the Spring reactive stack provides a higher-level interface for building reactive applications, relying on the familiar Spring projects without being concerned with the underlying implementation of the reactive streams.

8.2 Reactive servers with Spring WebFlux and Spring Data R2DBC

So far we have worked on Catalog Service, a non-reactive (or *imperative*) application, using Spring MVC and Spring Data JDBC. This section will teach you how to build a reactive web application (Order Service) using Spring WebFlux and Spring Data R2DBC. Order Service will provide functionality for purchasing books. Like Catalog Service, it will expose a REST API and store data in a PostgreSQL database. Unlike Catalog Service, it will use the reactive programming paradigm to improve scalability, resilience, and cost-effectiveness.

You'll see that the principles and patterns you learned in the previous chapters also apply to reactive applications. The main difference is that we'll shift from implementing the business logic in an imperative way to building reactive streams that are processed asynchronously.

Order Service will also interact with Catalog Service through its REST API to fetch details about books and check their availability. That will be the focus of section 8.3. Figure 8.4 shows the new components of the system.